

# Übungsblatt 1: Bare-Metal, Polling und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

## Musterlösung

### Zielsetzung

In dieser ersten Laborübung machen Sie sich mit der Entwicklungsumgebung und der Hardware vertraut. Sie lernen die praktischen Grenzen von klassischen Polling-Architekturen (Super-Loops) kennen, insbesondere das *Heavy-Load-Problem*. Anschließend lösen Sie dieses Problem, indem Sie die zeitkritischen Aufgaben in deterministische Hardware-Interrupts (Foreground/Background-Architektur) auslagern. Als Hardware dient das Dual-Core-Board **STM32H755**.

---

### Aufgabe 1: Hello World (LED Blink)

Um die Toolchain zu überprüfen, erstellen wir ein einfaches Bare-Metal-Programm auf dem Cortex-M4-Kern.

1. Legen Sie ein neues Projekt für das Board **STM32H755ZI-Q** in der STM32CubeIDE an.
2. Konfigurieren Sie eine LED (z. B. an Pin **PE1**) als `GPIO_Output` und weisen Sie diese dem Cortex-M4 zu (*Pin Context Assignment*).
3. Generieren Sie den Code.
4. Lassen Sie die LED in der `main.c` des CM4-Projekts im Abstand von 500 ms blinken. Nutzen Sie dafür die blockierende Funktion `HAL_Delay()`.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Musterlösung Aufgabe 1

Das Projekt wird generiert und der Code ausschließlich in den dafür vorgesehenen `USER CODE`-Blöcken eingefügt, damit er beim erneuten Generieren nicht gelöscht wird.

```

/* USER CODE BEGIN 3 */
while (1) {
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    HAL_Delay(500); /* blockiert 500 ms */
}
/* USER CODE END 3 */

```

---

## Aufgabe 2: Das Polling-Desaster

Wir provozieren nun einen Timing-Fehler, der zeigt, warum Polling-Loops in der Industrie oft ungeeignet sind.

1. Schreiben Sie eine einfache *Heavy-Load*-Funktion (z. B. eine sinnlose Zählschleife `Heavy_Load(800000)`), die CPU-Last und damit Ausführungszeit simuliert.
2. Rufen Sie diese Funktion in Ihrer `while(1)`-Schleife direkt vor dem LED-Toggle auf.
3. Flashen Sie den Code und beobachten Sie die LED. Erklären Sie, warum das 500-ms-Timing nun zerstört ist.

### Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

### Musterlösung Aufgabe 2

Hilfsfunktion (in einem USER CODE-Block, z. B. USER CODE BEGIN 0; Parameter als `volatile`, damit der Compiler die Schleife nicht wegoptimiert):

```

void Heavy_Load(volatile uint32_t n)
{
    while (n-- > 0) {
        /* absichtlich leer: reine CPU-Last */
    }
}

```

Schleife in main:

```

/* USER CODE BEGIN 3 */
while (1) {
    Heavy_Load(800000); /* CPU-Last simulieren */
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    HAL_Delay(500);
}
/* USER CODE END 3 */

```

**Erklärung des Polling-Desasters:** In einer Super-Loop (Polling) liegen Hintergrundarbeit und zeitkritische Aktion *sequenziell* hintereinander. Die unbekannte und variable Laufzeit von `Heavy_Load()` addiert sich zur Wartezeit von `HAL_Delay(500)`. Die CPU ist mit Rechnen beschäftigt und erreicht den Toggle-Befehl zu spät. Latenz und Periodizität werden unvorhersagbar (erhöhter Jitter). Das illustriert das klassische *Heavy-Load-Problem* einer Super-Loop-Architektur.

---

### Aufgabe 3: Rettung durch den Timer

Wir stellen den Determinismus durch Hardware-Präemption wieder her.

1. Aktivieren Sie im Device Configuration Tool (CubeMX) den Timer **TIM2** und wählen Sie als *Clock Source* „Internal Clock“.
2. Setzen Sie im Reiter *NVIC Settings* den Haken bei „TIM2 global interrupt“, damit die Hardware den IRQ zur CPU weiterleitet.
3. Berechnen und setzen Sie die Werte für Prescaler (PSC) und Auto-Reload Register (ARR), sodass der Timer exakt alle 500 ms (2 Hz) überläuft. Nutzen Sie die Formel

$$f_{\text{UPD}} = \frac{f_{\text{CLK}}}{(PSC + 1)(ARR + 1)}$$

Lesen Sie den tatsächlichen Timer-Takt  $f_{\text{CLK}}$  in der CubeMX-*Clock Configuration* ab (nicht raten!).

4. Generieren Sie den Code. Starten Sie den Timer in der `main()` mit `HAL_TIM_Base_Start_IT(&htim2)`.
5. Entfernen Sie den Toggle-Befehl und das Delay aus der `while(1)`-Schleife (dort verbleibt nur `Heavy_Load()`) und verlagern Sie das Umschalten der LED in die Timer-Callback-Funktion `HAL_TIM_PeriodElapsedCallback`.

### Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

### Musterlösung Aufgabe 3

#### 1. Hardware-Konfiguration (Beispielrechnung):

Angenommen, der Timer-Takt beträgt  $f_{\text{CLK}} = 200 \text{ MHz}$  (bitte am eigenen Projekt in CubeMX verifizieren; bei APB-Prescaler  $\neq 1$  verdoppelt der STM32 oft den Timer-Takt). Für  $f_{\text{UPD}} = 2 \text{ Hz}$  muss  $(PSC + 1)(ARR + 1) = 100\,000\,000$  gelten.

Beispiel:  $PSC = 19999$  und  $ARR = 4999$  ergeben  $(19999 + 1)(4999 + 1) = 20000 \cdot 5000 = 100\,000\,000$ , also  $f_{\text{UPD}} = 200\text{ MHz}/100\text{ MHz} = 2\text{ Hz}$  (Toggle alle 500 ms).

## 2. Main-Setup und Hintergrundschleife:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
/* USER CODE END 2 */

/* USER CODE BEGIN 3 */
while (1) {
    Heavy_Load(800000); /* Hintergrund-Last */
}
/* USER CODE END 3 */
```

## 3. Interrupt-Callback (typischerweise am Ende von main.c):

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); /* zeitkritische Arbeit */
    }
}
/* USER CODE END 4 */
```

**Fazit:** Dies entspricht der Foreground/Background-Architektur. Die ISR (Foreground) präemptiert die belastete Main-Loop (Background). Das Blinken ist wieder deterministisch — unabhängig von der CPU-Last im Hintergrund. Im Gegensatz zum Polling läuft die zeitkritische Aktion *neben* der Last, nicht *dahinter*.

---

## Aufgabe 4 (Bonus): Messung am Oszilloskop

In der Industrie wird Timing nicht nur berechnet, sondern physikalisch gemessen.

1. Konfigurieren Sie einen freien Pin (z. B. **PA0**) als **GPIO\_Output** und weisen Sie ihn ebenfalls dem Cortex-M4 zu (*Pin Context Assignment*).
2. Messen Sie die Ausführungszeit Ihrer CPU-Last, indem Sie den Pin vor **Heavy\_Load()** auf HIGH und direkt danach auf LOW setzen. Messen Sie die Pulsweite am Oszilloskop.
3. Vergleichen Sie den Jitter (die Schwankung der Ausführungszeitpunkte) der LED-Flanken in der Main-Loop (Aufgabe 2) mit dem Jitter der LED-Flanken im Timer-Interrupt (Aufgabe 3).

## Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

### Musterlösung Aufgabe 4 (Bonus)

Messaufbau für die Laufzeit von `Heavy_Load()`:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);  
Heavy_Load(800000);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
```

### Erwartetes Messergebnis:

- **Main-Loop (Aufgabe 2):** Die LED-Periodizität schwankt sichtbar (hoher Jitter), weil sich die variable Dauer von `Heavy_Load()` zum Delay addiert. Compiler-Optimierung und Cache können die Last zusätzlich streuen.
- **Timer-ISR (Aufgabe 3):** Die LED-Flanken sind sehr stabil (Jitter  $\approx 0$ ), weil der NVIC die CPU hardwaregesteuert auf Basis des Quarztakts unterbricht — unabhängig von der Hintergrundlast.

Die Pulsweite auf PA0 misst die *Ausführungszeit* der Lastfunktion (Beitrag zur WCET-Schätzung); der LED-Vergleich zeigt den Unterschied zwischen Software-Polling und Hardware-Präemption.